Java Synchronization Notes

1. Every Java Object Has a Lock

Every Java object contains a monitor (lock).

Example:

```
Object obj = new Object();
String str = "Hello";
BankAccount account = new BankAccount();
```

All of these can be used as locks:

```
synchronized(obj)
synchronized(str)
synchronized(account)
```

---

2. synchronized(this)

Example:

```
class BankAccount {

    public void withdraw() {

        synchronized(this) {

        }
    }

    public void updateProfile() {

        synchronized(this) {

        }
    }
}
```

Lock Used:

```
this = BankAccount Object
```

Memory Diagram

```
                    HEAP


          ┌──────────────────────┐
          │     BankAccount      │
          │----------------------│
          │ balance              │
          │ profile              │
          │                      │
          │ 🔒 Object Lock       │
          └──────────────────────┘
```

Only ONE lock exists.

Scenario A

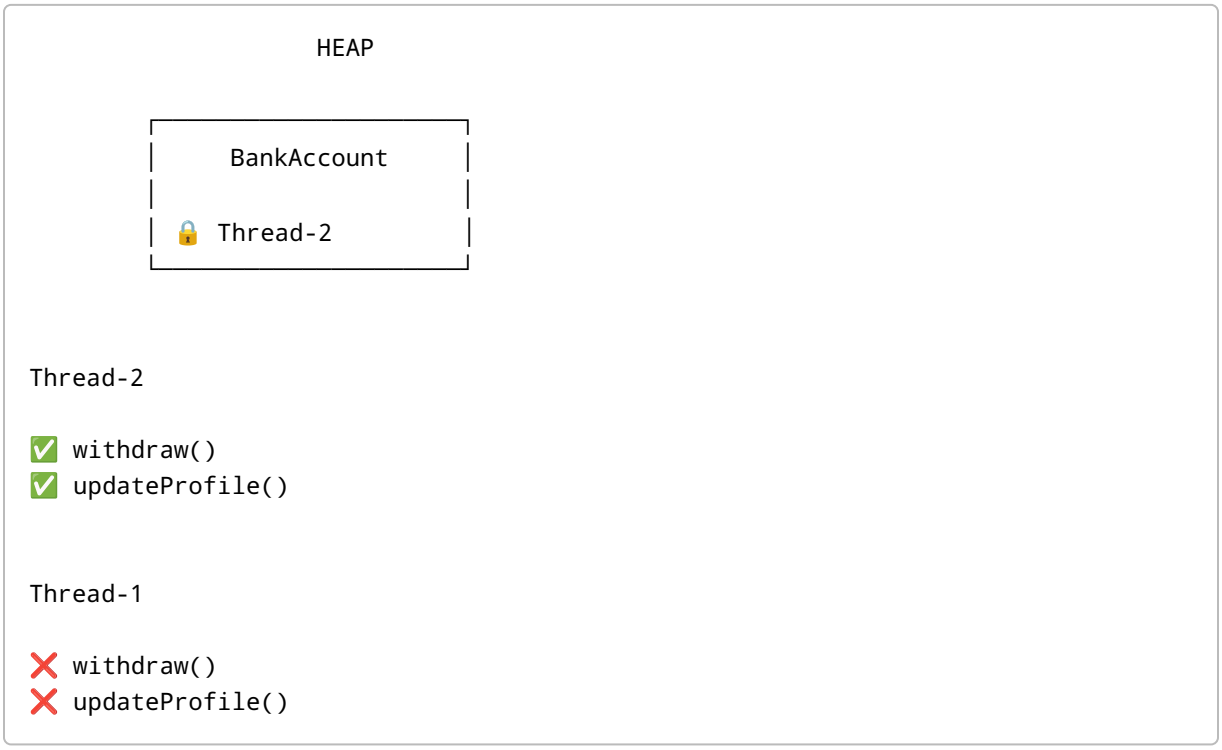
Thread-1 acquires lock first.

```
                    HEAP


          ┌──────────────────────┐
          │     BankAccount      │
          │                      │
          │ 🔒 Thread-1          │
          └──────────────────────┘


Thread-1

✅ withdraw()
✅ updateProfile()


Thread-2
```

❌ withdraw()
❌ updateProfile()

Scenario B

Thread-2 acquires lock first.



Mental Model

Entire object locked.

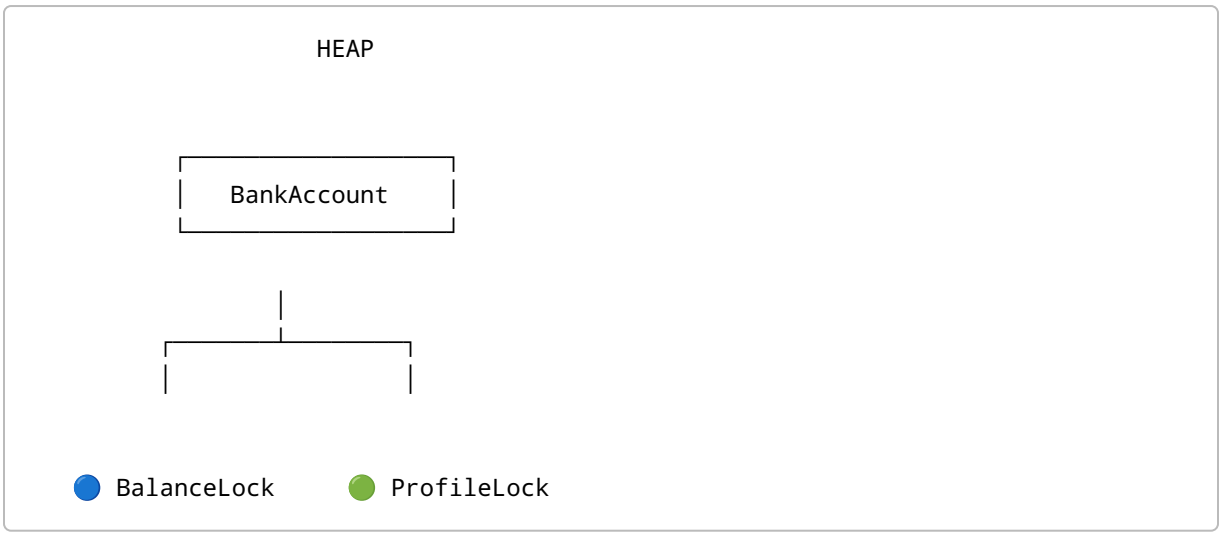
---

3. Dedicated Lock Objects

Example:

```
class BankAccount {

    private final Object balanceLock = new Object();

    private final Object profileLock = new Object();

    public void withdraw() {

        synchronized(balanceLock) {

        }
    }

    public void updateProfile() {

        synchronized(profileLock) {

        }
    }
}
```

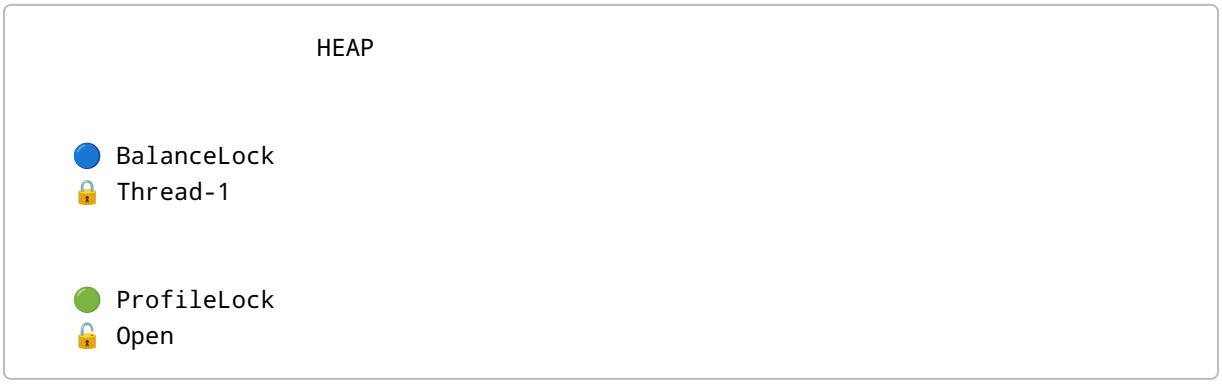
Memory Diagram



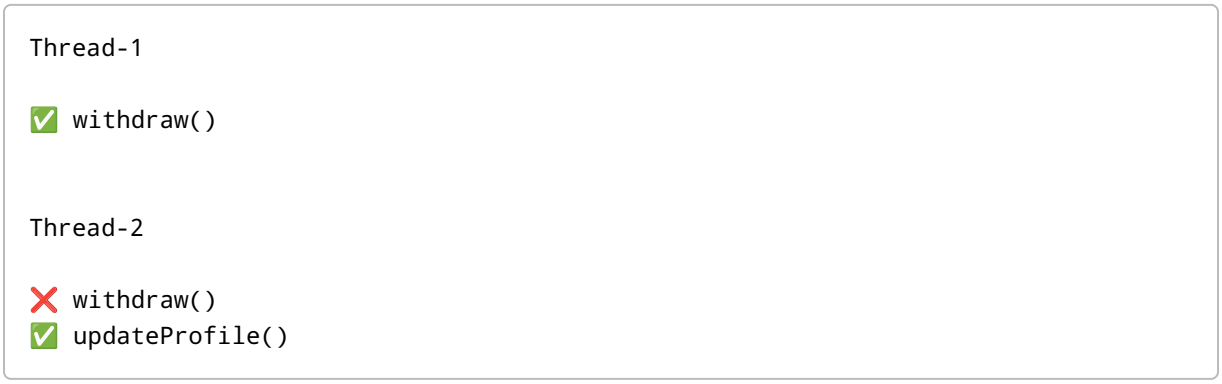
Now there are TWO locks.

Scenario A

Thread-1 owns BalanceLock



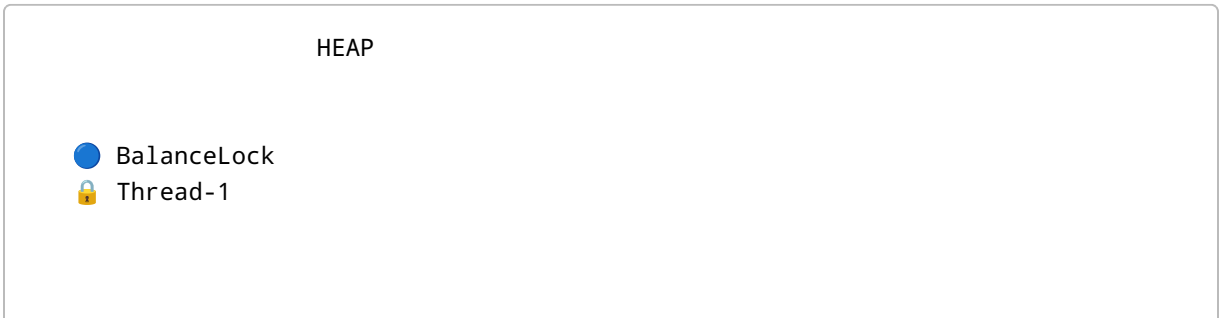
Access:



Scenario B

Thread-1 owns BalanceLock

Thread-2 owns ProfileLock



```
    🟢 ProfileLock
    🔒 Thread-2
```

Access:

```
Thread-1

✅ withdraw()
❌ updateProfile()


Thread-2

❌ withdraw()
✅ updateProfile()
```

Both work simultaneously.

Scenario C

Thread-2 owns both locks

```
                    HEAP


    🔵 BalanceLock
    🔒 Thread-2


    🟢 ProfileLock
    🔒 Thread-2
```

Access:

```
Thread-2

✅ withdraw()
✅ updateProfile()


Thread-1
```

```
❌ withdraw()
❌ updateProfile()
```

Mental Model

```
                HOUSE


      🔵 Kitchen Key
      🟢 Bedroom Key


Thread-1 → Kitchen

Thread-2 → Bedroom
```

Both can work at the same time.

---

4. Reentrant Locking

Example:

```
public void withdraw() {

    synchronized(this) {

        updateProfile();
    }
}
```

```
public void updateProfile() {

    synchronized(this) {

    }
}
```

Question:

Can Thread-1 call updateProfile() while already owning the same lock?

Answer:

YES

Reason:

Java locks are reentrant.

Visualization

Step 1

```
Thread-1

withdraw()

Lock Count = 1
```

Step 2

```
Thread-1

withdraw()
      |
      └── updateProfile()

Lock Count = 2
```

Step 3

```
Exit updateProfile()

Lock Count = 1
```

Step 4

```
Exit withdraw()

Lock Count = 0
```

```
🔓 Lock Released
```

Final Comparison

synchronized(this)

```
ONE OBJECT
ONE LOCK
ONE OWNER
```

```
BankAccount

      🔒

Thread-1 ✅
Thread-2 ❌
```

Dedicated Locks

```
MULTIPLE OBJECTS
MULTIPLE LOCKS
MULTIPLE OWNERS
```

```
BalanceLock  🔒 Thread-1

ProfileLock  🔒 Thread-2
```

Both threads can work simultaneously.

Interview Summary

synchronized(this)

→ Locks the entire object.

Dedicated Lock Object

→ Locks only the resource that needs protection.

Use dedicated locks when different resources should be accessed independently.